

Capítulo 5

Construcción de la gramática producto de una gramática de concatenación de rango y un lenguaje regular finito

La literatura provee construcciones de la intersección con un lenguaje regular para las gramáticas libres del contexto y las de concatenación de rango simple [1, 2]. En las gramáticas de concatenación de rango, una misma variable puede aparecer más de una vez en el lado derecho de una cláusula. Esta no linealidad introduce una dificultad que las construcciones anteriores no contemplan. Este capítulo aporta una construcción que, a partir de una gramática de concatenación de rango G y un autómata finito acíclico A , produce una nueva gramática de concatenación de rango G' , la **gramática producto**¹, cuyo lenguaje es exactamente $L(G) \cap L(A)$.

La construcción se apoya en una noción de rango adaptada al autómata, que se introduce en la sección siguiente.

5.1. El rango sobre un autómata

En una RCG el reconocimiento se hace por rangos; en un autómata, por estados. Para reconocer una cadena con ambos hay que coordinarlos: de cada rango se necesita saber en qué estado queda el autómata al empezar y al terminar

¹El término se usa por analogía con el autómata producto, la construcción estándar para la intersección de dos lenguajes regulares [4].

de leer su subcadena. Pero el rango (i, j) es un par de posiciones en una cadena fija, ajeno a los estados del autómata.

Hace falta, entonces, una noción de rango ligada a los estados del autómata. Para ello, cada subcadena se caracteriza por el par de estados entre los que transita el autómata al leerla. Para fijar ese par hace falta saber en qué estado termina el autómata al comenzar en un estado y leer una cadena, lo que da lugar a la siguiente definición.

Definición 5.1. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito determinista. La función de transición δ se extiende a cadenas mediante la **función de transición extendida** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, que se define por

$$\hat{\delta}(q, \varepsilon) = q \quad \text{y} \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

para $w \in \Sigma^*$ y $a \in \Sigma$ [4]. Así, $\hat{\delta}(q, u)$ es el estado en el que el autómata termina al comenzar en el estado q y leer la cadena u .

En los ejemplos de este capítulo se retoma la modelación del capítulo ?? de la fórmula $x_1 \wedge (x_2 \vee x_1)$, cuya secuencia de ocurrencias es $x_1 x_2 x_1$. Su autómata booleano A se muestra en la figura ?? . Es finito y acíclico: reconoce las cadenas de longitud tres que hacen verdadera la fórmula cuando cada ocurrencia se trata por separado. Su estado inicial es q_0 , su único estado de aceptación es V , y su lenguaje es $L(A) = \{101, 110, 111\}$.

Por ejemplo, al leer 101 sobre A se extiende $\hat{\delta}$ símbolo a símbolo,

$$\hat{\delta}(q_0, 1) = q_1, \quad \hat{\delta}(q_0, 10) = q_2 \quad \text{y} \quad \hat{\delta}(q_0, 101) = V,$$

un estado de aceptación; para la cadena 100, en cambio, se obtiene $\hat{\delta}(q_0, 100) = F$. La figura 5.1 muestra estos cuatro caminos en A .

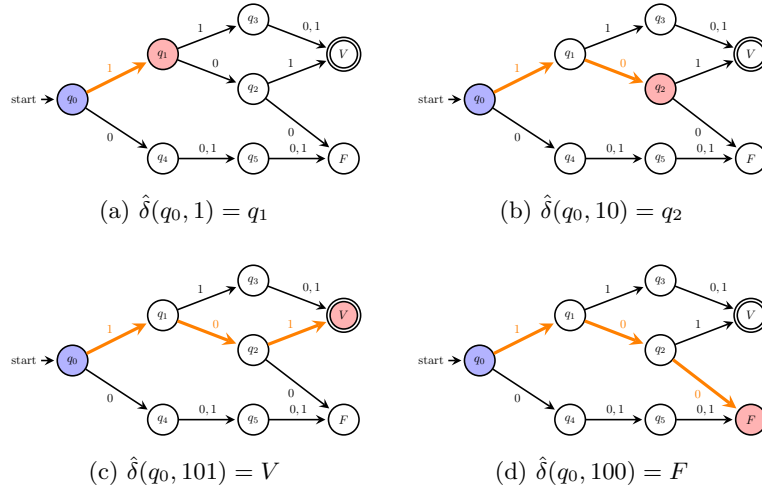


Figura 5.1: Caminos en A de los cuatro ejemplos de $\hat{\delta}$. En cada uno, el estado de partida q_0 va en azul y el de llegada en rojo; las transiciones resaltadas forman el camino, a lo largo del cual se lee la cadena correspondiente.

La función $\hat{\delta}$ determina, para cada cadena y cada estado de partida, el estado en que el autómata termina, lo que permite definir el rango sobre el autómata como un par de estados y precisar cuándo una cadena lo realiza.

Definición 5.2. Un **rango sobre** A es un par de estados $(q, q') \in Q \times Q$. Una cadena u **realiza** el rango (q, q') si $\hat{\delta}(q, u) = q'$.

Un rango sobre A describe a la vez todas las cadenas que lo realizan. Cuando a una variable se le asocia el rango (q, q') , la gramática debe derivar la subcadena correspondiente, que a la vez debe realizar ese rango.

En el ejemplo anterior, al leer 101 se recorre en A el camino $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2 \xrightarrow{1} V^2$. Así, el primer símbolo realiza (q_0, q_1) ; el segundo, (q_1, q_2) ; el tercero, (q_2, V) ; la subcadena 10 realiza (q_0, q_2) ; y la cadena entera 101 realiza (q_0, V) , que parte del estado inicial y termina en el de aceptación.

En la sección siguiente, las nociones planteadas para las cadenas se generalizan al caso de los argumentos.

5.2. El rango de un argumento sobre un autómata

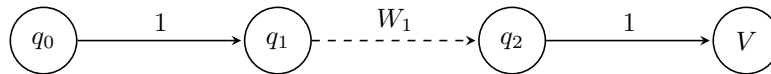
La sección anterior estableció cuándo una cadena realiza un rango sobre A . Un argumento de una cláusula es una secuencia de terminales y variables, y las variables no son símbolos que el autómata pueda leer. Hace falta entonces decir qué rango sobre A le corresponde a un argumento de esa forma.

Definición 5.3. Sea $\alpha = \alpha_1 \cdots \alpha_n$ un argumento, donde cada $\alpha_i \in T \cup V$. Una **asignación de rangos sobre** A para α es una función ρ que asigna a cada α_i un rango sobre A , $\rho(\alpha_i) = (q_i, q'_i)$, con dos condiciones:

1. $q'_i = q_{i+1}$ para $1 \leq i < n$;
2. si α_i es un terminal a , entonces $\delta(q_i, a) = q'_i$.

Cuando existe una asignación así, el argumento α realiza el rango sobre A (q_1, q'_n) .

Por ejemplo, considérese el argumento $W_0 \mid W_1 \mid W_2$ del lado izquierdo de la cláusula $Eq_{x_1}(W_0 \mid W_1 \mid W_2) \rightarrow L_0(W_0) L_1(W_1) L_0(W_2)$ de G_{cons} , que reconoce las cadenas de longitud tres con 1 en las dos posiciones de x_1 . Sobre la cadena consistente 101, con $W_0 = \varepsilon$, $W_1 = 0$ y $W_2 = \varepsilon$, la asignación de rangos sobre A es la siguiente, con W_0 y W_2 vacíos en los extremos:



²Recuérdese que en la notación de camino del capítulo ?? estados y símbolos se alternan.

Esta asignación cumple las dos condiciones de la definición de asignación de rangos sobre A . La primera, que el final de cada rango sea el inicio del siguiente, se ve al recorrer los rangos: W_0 realiza (q_0, q_0) ; el primer 1, (q_0, q_1) ; W_1 , (q_1, q_2) ; el segundo 1, (q_2, V) ; y W_2 , (V, V) . La segunda, que cada terminal a con rango (q_i, q'_i) cumpla $\delta(q_i, a) = q'_i$, la satisfacen los dos terminales 1: $\delta(q_0, 1) = q_1$ y $\delta(q_2, 1) = V$. El argumento realiza entonces (q_0, V) , que parte de q_0 y termina en el estado de aceptación V .

Con la cláusula del símbolo 0, $Eq_{x_1}(W_0 0 W_1 0 W_2)$, se da el caso contrario. Al leer la cadena consistente 000 se recorre el camino $q_0 0 q_4 0 q_5 0 F$, de modo que el argumento realiza (q_0, F) , que no termina en el estado de aceptación: 000 es consistente, pero hace falsa la fórmula.

La sección siguiente presenta la construcción de la gramática producto.

5.3. Construcción de la gramática producto

En esta sección se desarrolla la construcción de la gramática producto. Primero se presenta el algoritmo. A continuación se impone una restricción sobre la gramática de entrada. Después se establece una condición para que las ocurrencias de una misma variable reciban el mismo rango. Luego se da el pseudocódigo. Por último se demuestra su correctitud y se analizan su tamaño y la complejidad de construirla.

5.3.1. Algoritmo

El objetivo es producir una gramática $G' = (N', T, V, P', S')$ que reconozca las cadenas que G reconoce y A acepta. Una cadena w pertenece a $L(A)$ si el rango sobre A que w realiza parte del estado inicial y termina en un estado final, así que G' no solo debe reconocer las cadenas de $L(G)$, sino reflejar también qué rango sobre A realiza cada argumento de cada predicado durante la derivación. Para reflejarlo, los predicados de G' se diferencian según el rango sobre A que realiza cada uno de sus argumentos: cada predicado de G da lugar a un predicado de G' por cada vector de rangos sobre A .

Definición 5.4. Sean B un predicado de G de aridad k y $\vec{q} = ((q_1, q'_1), \dots, (q_k, q'_k))$ un vector de k rangos sobre A , uno por cada argumento de B . El par $(B, \vec{q})$ se llama **predicado indexado** y se denota $B[\vec{q}]$.

Por ejemplo, el autómata A tiene ocho estados, así que cada predicado de G_{cons} da lugar a hasta sesenta y cuatro predicados indexados, uno por cada par de estados. En el reconocimiento de 101 aparecen $S[(q_0, V)]$, $Eq_{x_1}[(q_0, V)]$ y $Eq_{x_2}[(q_0, V)]$, junto con predicados indexados de L_0 y L_1 .

El predicado inicial de G' es un predicado S' de aridad 1. El conjunto de predicados N' consta de los predicados indexados y del predicado inicial S' .

Las cláusulas de G' se generan a partir de las cláusulas de G y de las asignaciones de rangos sobre A a sus ocurrencias de símbolos. Para cada cláusula

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$$

de G y cada asignación que cumpla las condiciones de la sección 5.2, indexando cada predicado B_j por el vector $\vec{q}_j$ de los rangos de sus argumentos se obtiene una **cláusula indexada**³:

$$B_0[\vec{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\vec{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\vec{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m}).$$

Por ejemplo, en la cláusula $L_1(1X) \rightarrow L_0(X)$ de G_{cons} , con el terminal 1 en el rango (q_0, q_1) , pues $\delta(q_0, 1) = q_1$, y la variable X en (q_1, q_1) , el argumento $1X$ realiza (q_0, q_1) . La cláusula indexada que se obtiene es

$$L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X).$$

La construcción genera una cláusula indexada por cada asignación de rangos que cumpla las condiciones, de modo que cada cláusula de G_{cons} da lugar a muchas.

Cuando el lado derecho de una cláusula de G es vacío, solo se indexa el lado izquierdo: la cláusula $B(\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$ produce las cláusulas indexadas $B[\vec{q}](\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$, una por cada asignación de rangos a sus argumentos. El argumento ε realiza solo el rango (q, q) para cada estado q , así que la cláusula $L_0(\varepsilon) \rightarrow \varepsilon$ produce ocho cláusulas en G' , una por cada estado del autómata:

$$\begin{aligned} L_0[(q_0, q_0)](\varepsilon) &\rightarrow \varepsilon, & L_0[(q_1, q_1)](\varepsilon) &\rightarrow \varepsilon, \\ L_0[(q_2, q_2)](\varepsilon) &\rightarrow \varepsilon, & L_0[(V, V)](\varepsilon) &\rightarrow \varepsilon, \end{aligned}$$

y de igual forma para los estados q_3, q_4, q_5 y F .

El conjunto de cláusulas P' consta de las cláusulas anteriores y de las del predicado inicial S' , que dependen de la condición de aceptación de A : una cadena se acepta cuando el rango que realiza parte del estado inicial q_0 y termina en un estado final. Por eso la construcción solo usa los predicados indexados $S[(q_0, q_F)]$ con $q_F \in F$, y, para cada uno, G' tiene una cláusula

$$S'(X) \rightarrow S[(q_0, q_F)](X),$$

donde S es el predicado inicial de G . Una cadena w pertenece a $L(G')$ si $S'(w)$ se deriva a la cadena vacía.

En el ejemplo de G_{cons} sobre A , $F = \{V\}$ ⁴, y el rango (q_0, V) parte del estado inicial y termina en el estado de aceptación. El predicado inicial S' tiene la cláusula:

$$S'(X) \rightarrow S[(q_0, V)](X).$$

La construcción conserva todas las cláusulas que produce, tanto las que provienen de las cláusulas de G como las del predicado inicial S' . Las que participan en el reconocimiento de una cadena se determinan durante la derivación, no durante la construcción.

³Una cláusula indexada es una cláusula de la gramática producto G' , formada por predicados indexados.

⁴Aquí F es el conjunto de estados de aceptación del autómata, no el estado F del autómata booleano.

La construcción produce los predicados N' , las cláusulas P' y el predicado inicial S' , y mantiene los terminales T y las variables V de G . Los cinco componentes completan la tupla $G' = (N', T, V, P', S')$, con lo que la construcción de la gramática producto termina.

Para ilustrar el reconocimiento en la gramática producto G' , se describe la derivación de la cadena $101 \in L(G_{\text{cons}}) \cap L(A)$, el testigo de que la fórmula es satisfacible. Cada predicado se indexa por el rango que su argumento realiza en la lectura de 101 por A , cuyo camino es $q_0 \ 1 \ q_1 \ 0 \ q_2 \ 1 \ V$:

$$\begin{aligned}
S'(101) &\rightarrow S[(q_0, V)](101), \\
S[(q_0, V)](101) &\rightarrow Eq_{x_1}[(q_0, V)](101) \ Eq_{x_2}[(q_0, V)](101), \\
Eq_{x_1}[(q_0, V)](101) &\rightarrow L_0[(q_0, q_0)](\varepsilon) \ L_1[(q_1, q_2)](0) \ L_0[(V, V)](\varepsilon), \\
Eq_{x_2}[(q_0, V)](101) &\rightarrow L_1[(q_0, q_1)](1) \ L_1[(q_2, V)](1), \\
L_1[(q_1, q_2)](0) &\rightarrow L_0[(q_2, q_2)](\varepsilon), \\
L_1[(q_0, q_1)](1) &\rightarrow L_0[(q_1, q_1)](\varepsilon), \\
L_1[(q_2, V)](1) &\rightarrow L_0[(V, V)](\varepsilon),
\end{aligned}$$

y cada predicado indexado $L_0[(q, q)](\varepsilon)$ deriva en la cadena vacía por la cláusula terminal. La figura 5.2 muestra el árbol de derivación, con cada predicado indexado por el rango que realiza su argumento. El primer paso aplica la cláusula del predicado inicial S' ; el segundo, la cláusula no lineal de S , que asocia el mismo rango (q_0, V) a las dos ocurrencias de X . Como la derivación de $S'(101)$ termina en la cadena vacía, G' reconoce 101.

La derivación de 101 usa solo unos pocos de los predicados y cláusulas de G' . La construcción genera muchos más que no aparecen en el reconocimiento de ninguna cadena de $L(G_{\text{cons}}) \cap L(A)$: los predicados indexados cuyo rango no parte de q_0 ni termina en V , como $Eq_{x_1}[(q_0, F)]$, quedan sin uso, junto con las cláusulas que los tienen en el lado izquierdo.

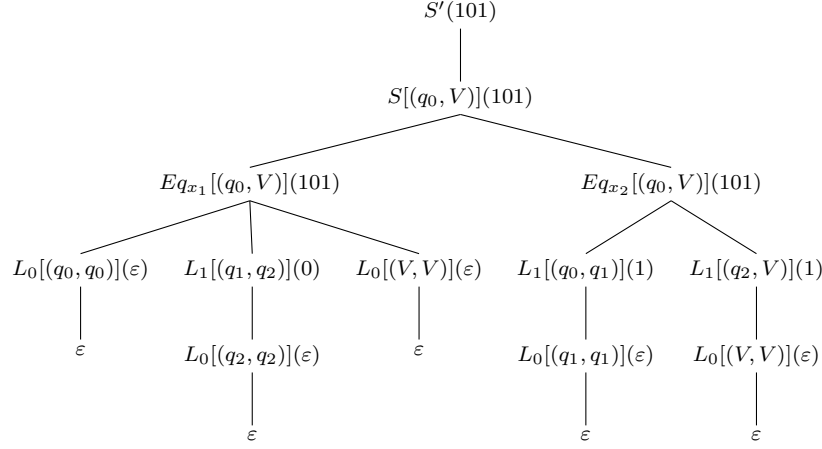


Figura 5.2: Árbol de derivación de 101 en la gramática producto G' . Cada predicado va indexado por el rango que realiza su argumento; las hojas ε corresponden a las cláusulas terminales $L_0[(q, q)](\varepsilon) \rightarrow \varepsilon$.

Por conveniencia, la construcción anterior se restringe a una clase de gramáticas de entrada que evita casos aparte. La sección siguiente la precisa.

5.3.2. Restricción sobre la gramática de entrada

Cada predicado se indexa por los rangos sobre A de sus argumentos. El lado derecho de una cláusula fija los rangos de las variables que contiene. Si una variable apareciera en el lado izquierdo pero no en el lado derecho, el lado derecho no fijaría su rango, y habría que tratarla como un caso aparte.

Para evitarlo, las gramáticas de entrada se restringen a las que cumplen una propiedad sobre sus cláusulas: en cada cláusula, toda variable del lado izquierdo aparece también en el lado derecho.

Definición 5.5. Una gramática de concatenación de rango es **no borrante** si en cada cláusula toda variable del lado izquierdo aparece también en el lado derecho.

Se pide además que los argumentos del lado derecho sean variables, de modo que cada variable del lado derecho sea un argumento por sí sola.

Definición 5.6. Una gramática de concatenación de rango es **no combinatoria** si en cada cláusula los argumentos del lado derecho son variables.

Por ejemplo, las cláusulas de G_{cons} cumplen ambas: en $Eq_{x_1}(W_0 \mid W_1 \mid W_2) \rightarrow L_0(W_0) L_1(W_1) L_0(W_2)$ las variables W_0, W_1, W_2 del lado izquierdo aparecen en el lado derecho, cada una como un argumento por sí sola, y $L_0(\varepsilon) \rightarrow \varepsilon$ no tiene variables. Una cláusula como $A(XY) \rightarrow B(X)$ no cumple la condición de no borrante, pues Y figura en el lado izquierdo y falta en el lado derecho, y

$A(X) \rightarrow B(X1)$ no cumple la de no combinatoria, pues el argumento del lado derecho $X1$ no es una variable.

Exigir ambas propiedades no descarta ningún lenguaje: toda gramática de concatenación de rango admite una equivalente que las cumple [3, 5].

Además de la restricción sobre la gramática de entrada, hace falta una condición sobre los rangos que se asignan, que se detalla a continuación.

5.3.3. Variables repetidas en una cláusula

En las gramáticas de concatenación de rango, una misma variable puede aparecer varias veces en una cláusula. En la construcción de la sección 5.3.1, cada argumento del lado derecho se indexa por separado, sin que se fije el mismo rango para las ocurrencias de una misma variable, que representan una sola cadena. Sin una condición que lo imponga, se les pueden asignar rangos distintos, y entonces G' reconocería cadenas que no están en la intersección.

Por eso es necesario añadir la siguiente condición: en cada cláusula, a todas las ocurrencias de una variable se les asigna el mismo rango. Más formalmente, si una variable X aparece en dos argumentos de la cláusula con asignaciones de rangos ρ y ρ' , entonces $\rho(X) = \rho'(X)$.

Por ejemplo, la cláusula no lineal de G_{cons} ,

$$S(X) \rightarrow Eq_{x_1}(X) Eq_{x_2}(X),$$

tiene la variable X en tres argumentos: el del lado izquierdo y los dos del lado derecho. Sobre A , la cadena 000 es consistente, pero hace falsa la fórmula: realiza (q_0, F) , que no termina en un estado de aceptación.

Sin esta condición existe la cláusula

$$S[(q_0, V)](X) \rightarrow Eq_{x_1}[(q_0, F)](X) Eq_{x_2}[(q_0, F)](X),$$

con el predicado del lado izquierdo indexado en (q_0, V) y los del lado derecho en (q_0, F) . Como 000 realiza (q_0, F) y es consistente, los dos predicados Eq del lado derecho se derivan en la cadena vacía; con la cláusula $S'(X) \rightarrow S[(q_0, V)](X)$, la gramática G' reconocería 000, que está fuera de la intersección.

Con la condición, las tres ocurrencias de X llevan el mismo rango, así que la única cláusula cuyo lado izquierdo se indexa en (q_0, V) es

$$S[(q_0, V)](X) \rightarrow Eq_{x_1}[(q_0, V)](X) Eq_{x_2}[(q_0, V)](X).$$

Como 000 realiza (q_0, F) y no (q_0, V) , el predicado $Eq_{x_1}[(q_0, V)](000)$ no se deriva en la cadena vacía, así que 000 queda fuera del lenguaje de G' , como debería ser.

La repetición de variables es justo lo que separa a las RCG de las sRCG, y la condición no hace falta para estas últimas: en una sRCG cada variable aparece una sola vez en el lado derecho, así que no hay varias ocurrencias que coordinar. Imponerla es lo que permite tratar RCG y no solo sRCG.

Ya se presentaron la idea del algoritmo, la restricción sobre la gramática de entrada y la condición sobre las ocurrencias de una misma variable; con ellas se fija la construcción. A continuación se describe el pseudocódigo.

5.3.4. Pseudocódigo

El procedimiento PRODUCT-GRAMMAR recibe una gramática de concatenación de rango no combinatoria y no borrante $G = (N, T, V, P, S)$ y un autómata finito determinista acíclico $A = (Q, \Sigma, \delta, q_0, F)$; devuelve $G' = (N', T, V, P', S')$ con $L(G') = L(G) \cap L(A)$.

PRODUCT-GRAMMAR(G, A)

```

1   $P' = \emptyset$ 
2  for each clause  $c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow$ 
    $B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$  in  $P$ 
3      let  $\sigma_1, \sigma_2, \dots$  be the symbol occurrences of  $c$  and its empty arguments
4       $\ell =$  number of these
5      for each  $\mu = ((q_1, q'_1), \dots, (q_\ell, q'_\ell)) \in (Q \times Q)^\ell$ 
6          if CONSISTENT( $c, \mu$ )
7              for  $j = 0$  to  $m$ 
8                   $\vec{q}_j = (\text{RANGE}(\alpha_{j,1}, \mu), \dots, \text{RANGE}(\alpha_{j,k_j}, \mu))$ 
9                  add  $B_0[\vec{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow$ 
                      $B_1[\vec{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\vec{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m})$  to  $P'$ 
10 for each  $q \in F$ 
11     add  $S'(X) \rightarrow S[(q_0, q)](X)$  to  $P'$ 
12  $N' = \{ S' \} \cup \{ B' : B' \text{ occurs in } P' \}$ 
13 return  $(N', T, V, P', S')$ 

```

En la línea 2 se recorren las cláusulas de G . Para cada una, en las líneas 3 y 4 se cuentan los símbolos que aparecen en ella, con repeticiones, y sus argumentos vacíos. En la línea 5 se recorren todas las posibles asignaciones de rangos. De la línea 6 a la 9, con CONSISTENT se conservan solo las asignaciones en las que cada rango termina donde empieza el siguiente, cada terminal a con rango (q, q') cumple $\delta(q, a) = q'$, y todas las ocurrencias de una variable reciben el mismo rango; por cada una se indexa cada predicado por los rangos de sus argumentos con RANGE, y se agrega la cláusula a P' . En las líneas 10 y 11 se agregan las cláusulas del predicado inicial S' , una por cada estado final. En la línea 12 se agregan a N' el predicado S' y los predicados que aparecen en P' .

En CONSISTENT se comprueban las condiciones sobre μ :

CONSISTENT(c, μ)

```

1  for each non-empty argument  $\alpha = \alpha_1 \cdots \alpha_n$  of  $c$ , with  $\mu(\alpha_i) = (q_i, q'_i)$ 
2      for  $i = 1$  to  $n$ 
3          if  $i < n$  and  $q'_i \neq q_{i+1}$ 
4              return FALSE
5          if  $\alpha_i$  is a terminal  $a$  and  $\delta(q_i, a) \neq q'_i$ 
6              return FALSE
7  for each empty argument  $\alpha$  of  $c$ , with  $\mu(\alpha) = (q, q')$ 
8      if  $q \neq q'$ 
9          return FALSE
10 for each variable  $X$  of  $c$ 
11     if two occurrences of  $X$  have different ranges under  $\mu$ 
12         return FALSE
13 return TRUE

```

En las líneas 1–9 de CONSISTENT se comprueban las condiciones sobre los rangos de los argumentos; en las líneas 10–12, que las ocurrencias de una variable reciban el mismo rango.

RANGE(α, μ) es el rango que el argumento α realiza bajo μ :

RANGE(α, μ)

```

1  if  $\alpha = \varepsilon$ 
2      return  $\mu(\alpha)$ 
3  let  $\alpha = \alpha_1 \cdots \alpha_n$ , with  $\mu(\alpha_i) = (q_i, q'_i)$ 
4  return  $(q_1, q'_n)$ 

```

Para $\alpha = \alpha_1 \cdots \alpha_n$ es (q_1, q'_n) , el inicio del primer símbolo y el final del último; para $\alpha = \varepsilon$ es el rango (q, q) que μ le asigna, el rango que α realiza según la sección 5.2.

En la próxima subsección se demuestra que G' reconoce exactamente $L(G) \cap L(A)$.

5.3.5. Correctitud

En toda la sección G es no combinatoria y no borrante. La igualdad $L(G') = L(G) \cap L(A)$ se prueba por las dos inclusiones. La clave es una propiedad de los índices: una cláusula indexada solo reconoce cadenas que realizan los rangos con que se indexan sus argumentos. De ahí, A acepta toda cadena que G' reconoce; y, al borrar los índices, también la reconoce G . En el otro sentido, se parte de una derivación de G y se indexa cada predicado con los rangos sobre A que sus argumentos realizan al leer la cadena; el resultado es una derivación de G' .

Sea $B_0[\vec{q}_0](u_1, \dots, u_{k_0})$ el predicado indexado $B_0[\vec{q}_0]$ aplicado a las cadenas $u_1, \dots, u_{k_0}$, una por cada argumento, con $\vec{q}_0 = ((q_1, q'_1), \dots, (q_{k_0}, q'_{k_0}))$.

Lema 5.1. *Si $B_0[\vec{q}_0](u_1, \dots, u_{k_0})$ se deriva a ε en G' , entonces $B_0(u_1, \dots, u_{k_0})$ se deriva a ε en G .*

Demostración. Las cláusulas que intervienen en una derivación de $B_0[\vec{q}_0](u_1, \dots, u_{k_0})$ son cláusulas indexadas, cada una proveniente de una cláusula de P a la que solo se le agregan los índices de los predicados; el predicado S' no aparece, pues no figura en el lado derecho de ninguna cláusula. Al borrar los índices, cada cláusula indexada se reduce a su cláusula de P , y se obtiene una derivación de $B_0(u_1, \dots, u_{k_0})$ en G . $\square$

La hipótesis de que G es no combinatoria es clave en el segundo lema.

Lema 5.2. *Si $B_0[\vec{q}_0](u_1, \dots, u_{k_0})$ se deriva a ε en G' , entonces cada u_l realiza el rango (q_l, q'_l) .*

Demostración. La prueba es por inducción en la cantidad de pasos de la derivación de $B_0[\vec{q}_0](u_1, \dots, u_{k_0})$ a ε . La hipótesis de inducción es que el lema vale para toda derivación con menos pasos.

La derivación comienza con una cláusula de P' de lado izquierdo $B_0[\vec{q}_0]$, que proviene de una cláusula de P ,

$$c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m}),$$

y de una asignación μ de un rango a cada ocurrencia de símbolo de c . Recuérdese que esta asignación cumple tres condiciones. La primera es que en cada argumento el final del rango de un símbolo sea el inicio del siguiente. La segunda es que cada terminal a con rango (q, q') cumpla $\delta(q, a) = q'$. La tercera es que todas las ocurrencias de una variable reciban el mismo rango. La cláusula indexada que resulta es

$$B_0[\vec{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\vec{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\vec{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m}),$$

con (q_l, q'_l) el rango que el argumento $\alpha_{0,l}$ del lado izquierdo realiza bajo μ . La cláusula se instancia con una sustitución θ de las variables por subcadenas, de modo que $u_l = \theta(\alpha_{0,l})$.

En el caso base, el lado derecho de c es vacío ($m = 0$), de modo que

$$c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow \varepsilon,$$

y la derivación tiene un solo paso. Como el lado derecho es vacío y G es no borrante, el lado izquierdo no tiene variables, pues toda variable suya tendría que aparecer en el lado derecho. Así, cada argumento $\alpha_{0,l}$ es una cadena de terminales, y por la primera y la segunda condición $u_l = \alpha_{0,l}$ realiza (q_l, q'_l) .

En el paso inductivo, el lado derecho no es vacío, y cada predicado

$$B_j[\vec{q}_j](\theta(\alpha_{j,1}), \dots, \theta(\alpha_{j,k_j}))$$

se deriva a ε en menos pasos. Como G es no combinatoria, cada argumento del lado derecho es una sola variable X , con un único rango $\mu(X)$ en todas sus ocurrencias por la tercera condición, y el rango que ese argumento realiza es $\mu(X)$. Por la hipótesis de inducción, $\theta(X)$ realiza $\mu(X)$ para toda variable X del lado derecho; como G es no borrante, lo mismo vale para toda variable de c .

Sea entonces $\alpha_{0,l}$ un argumento del lado izquierdo. Por la segunda condición, cada terminal de $\alpha_{0,l}$ realiza el rango que μ le asigna; y para cada variable X , $\theta(X)$ realiza $\mu(X)$, que por la tercera condición es el rango de todas sus ocurrencias. Como el final de cada rango es el inicio del siguiente, por la primera condición, la concatenación de estas subcadenas, $u_l = \theta(\alpha_{0,l})$, realiza la concatenación de sus rangos, (q_l, q'_l) . Si $\alpha_{0,l} = \varepsilon$, su rango (q_l, q'_l) tiene $q_l = q'_l$, por ser el de un argumento vacío, y $u_l = \varepsilon$ lo realiza. $\square$

Con los dos lemas se obtiene el teorema.

Teorema 5.1. $L(G') = L(G) \cap L(A)$.

Demostración. Inclusión $L(G') \subseteq L(G) \cap L(A)$. Sea $w \in L(G')$; entonces $S'(w)$ se deriva a ε . Las únicas cláusulas de lado izquierdo S' son $S'(X) \rightarrow S[(q_0, q_F)](X)$ con $q_F \in F$, así que el primer paso es $S'(w) \rightarrow S[(q_0, q_F)](w)$ para algún $q_F \in F$, y $S[(q_0, q_F)](w)$ se deriva a ε . Por el lema 5.1, $S(w)$ se deriva a ε en G , esto es, $w \in L(G)$. Por el lema 5.2, w realiza (q_0, q_F) , es decir, $\hat{\delta}(q_0, w) = q_F$; como $q_F \in F$, $w \in L(A)$. Luego $w \in L(G) \cap L(A)$.

Inclusión $L(G) \cap L(A) \subseteq L(G')$. Sea $w = a_1 \cdots a_n \in L(G) \cap L(A)$. El camino de la lectura de w por A es $p_0 a_1 p_1 \cdots a_n p_n$, con $p_0 = q_0$ y $p_i = \delta(p_{i-1}, a_i)$; como $w \in L(A)$, $p_n \in F$. La subcadena en las posiciones i a j realiza el rango sobre A (p_i, p_j) .

Como $w \in L(G)$, hay una derivación de $S(w)$ a ε en G , en la que cada predicado se aplica a subcadenas de w en posiciones fijas. Al indexar cada predicado por los rangos sobre A de sus argumentos se obtiene una derivación en G' , porque cada cláusula indexada pertenece a P' . En efecto, sea

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$$

una cláusula instanciada en la derivación, y μ la asignación que a cada ocurrencia de símbolo le asigna el rango sobre A de la posición que ocupa. La asignación μ cumple las tres condiciones: los símbolos contiguos de un argumento ocupan posiciones contiguas, así que el final de cada rango es el inicio del siguiente, y un argumento vacío en la posición i ocupa el rango (p_i, p_i) ; para un terminal a en la posición $i+1$ se tiene $\delta(p_i, a) = p_{i+1}$; y las ocurrencias de una variable ocupan las mismas posiciones, luego reciben el mismo rango. Por tanto, la construcción produce la cláusula indexada, que pertenece a P' .

El lado izquierdo $S'(w)$ se indexa por el rango $(p_0, p_n) = (q_0, p_n)$. Como $p_n \in F$, la cláusula $S'(X) \rightarrow S[(q_0, p_n)](X)$ pertenece a P' , así que $S'(w)$ se deriva a ε y $w \in L(G')$. $\square$

La gramática producto es correcta, pero su tamaño puede ser exponencial en la entrada, como se ve a continuación.

5.3.6. Tamaño y complejidad

El tamaño de G' y el costo de construirla dependen del número de predicados $|N|$ y cláusulas $|P|$ de G , de los $|Q|$ estados de A , de la aridad máxima a y del máximo ℓ de ocurrencias de símbolo y argumentos vacíos por cláusula.

Cada predicado de aridad k se indexa con un vector de k rangos sobre A . Un rango es un par de estados, de modo que hay $|Q|^2$ rangos y $(|Q|^2)^k = |Q|^{2k}$ vectores posibles. Como la aridad máxima es a , por cada uno de los $|N|$ predicados hay a lo sumo $|Q|^{2a}$ predicados indexados. Con el predicado inicial,

$$|N'| = O(|N| |Q|^{2a}).$$

Para cada cláusula con ℓ_c ocurrencias de símbolo se recorren, en la línea 5 del procedimiento PRODUCT-GRAMMAR, las $|Q|^{2\ell_c}$ asignaciones de un rango a cada una, y por cada una que cumple las tres condiciones se agrega a lo sumo una cláusula indexada; las del predicado inicial son una por estado final. Así,

$$|P'| = O(|P| |Q|^{2\ell} + |F|) = O(|P| |Q|^{2\ell}).$$

Como los terminales son los de G , el tamaño de G' es

$$|G'| = |N'| + |T| + |P'| = O(|N| |Q|^{2a} + |P| |Q|^{2\ell} + |T|).$$

El costo de construir G' es el número de asignaciones recorridas por el $O(\ell_c)$ de verificar y procesar cada una:

$$O(|P| \ell |Q|^{2\ell}).$$

Los factores $|Q|^{2a}$ y $|Q|^{2\ell}$ son exponenciales en la aridad máxima a y en ℓ . Por eso el tamaño de G' es exponencial cuando a o ℓ no están acotados, y el costo de construirla, con solo el factor $|Q|^{2\ell}$, lo es cuando ℓ no está acotado. El valor de ℓ crece con la cantidad de variables de un mismo argumento del lado izquierdo, pues por cada una queda libre el rango con que se la indexa.

En la modelación del capítulo ??, G_{cons} es la gramática de consistencia de una fórmula con n ocurrencias de variable booleana y A su autómata booleano, con $|N| = O(n)$, $|P| = O(n)$, $|Q| = O(n^2)$ y $a = 1$. El número de predicados indexados es polinomial,

$$|N'| = O(n(n^2)^2) = O(n^5).$$

El número de cláusulas no lo es: la cláusula de Eq_v para una variable booleana con r ocurrencias,

$$Eq_v(W_0 b W_1 b \cdots b W_r) \rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \cdots L_{g_r}(W_r),$$

tiene $r + 1$ variables en su argumento del lado izquierdo, y r puede ser $O(n)$, de modo que $\ell = O(n)$. Entonces

$$|P'| = O(n(n^2)^{O(n)}) = 2^{O(n \log n)}.$$

El tamaño de la gramática producto de G_{cons} y el autómata booleano es

$$|G'| = |N'| + |T| + |P'| = O(n^5) + O(1) + 2^{O(n \log n)} = 2^{O(n \log n)}.$$

Construirla cuesta lo mismo:

$$O(|P| \ell |Q|^{2\ell}) = O(n^2 (n^2)^{O(n)}) = 2^{O(n \log n)}.$$

Las cláusulas de Eq_v son el término dominante.

En SATSRC el grado del polinomio con que se decide el vacío de la intersección crece con la aridad de la sRCG, como se ve en la subsección ?? del capítulo ?. La aridad $a = 1$ de G_{cons} se debe a su no linealidad: un mismo argumento aparece en varios predicados del lado derecho, y así la consistencia de cada variable booleana se reconoce sin aumentar la aridad. A cambio se pierde la tratabilidad del vacío de la intersección, polinomial con una sRCG y NP-completo con una RCG no lineal.

Construir G' sirve para decidir el vacío de la intersección, el problema del capítulo ?. Como $L(G') = L(G) \cap L(A)$, esa intersección es vacía si y solo si lo es $L(G')$, esto es, si el predicado inicial S' no es productivo. Para esa decisión solo importan los predicados y cláusulas productivos, y muchos de los indexados no lo son: no intervienen en el reconocimiento de ninguna cadena de $L(G_{\text{cons}}) \cap L(A)$, como se observó en la sección 5.3.1. En este trabajo no se da el costo de decidir ese vacío, sino solo el objeto G' ; limpiarlo de los predicados y cláusulas no productivos y analizar ese costo quedan como problemas abiertos.

En este capítulo se construyó la gramática producto G' de una RCG G y un autómata finito acíclico A , se demostró que $L(G') = L(G) \cap L(A)$, se caracterizaron su tamaño y el costo de construirla, exponenciales en el peor caso, y se mostró que muchos de los predicados y cláusulas de G' no son productivos. En el próximo capítulo se presentan las conclusiones y recomendaciones del trabajo.

Bibliografía

- [1] Yehoshua Bar-Hillel, Micha Perles, and Eli Shamir. On formal properties of simple phrase structure grammars. *Zeitschrift für Phonetik, Sprachwissenschaft und Kommunikationsforschung*, 14(2):143–172, 1961.
- [2] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [3] Pierre Boullier. Proposal for a natural language processing syntactic backbone. Technical Report RR-3342, INRIA-Rocquencourt, France, 1998.
- [4] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [5] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Cognitive Technologies. Springer, Berlin, Heidelberg, 2010.